

TASK 5: Real-Time Chat Application

Domain: Real-Time Systems | Level: Advanced

What is this project?

A multi-user chat application using Java Sockets. One program acts as the Server, another as the Client. Multiple clients can connect simultaneously and exchange messages in real time. This teaches client-server architecture and threading.

Key Concepts to Learn First

- `ServerSocket` – listens for connections on a port
- `Socket` – represents a connection between client and server
- `BufferedReader` / `PrintWriter` – for reading and writing over sockets
- Threads per client – each connected client gets its own thread
- Broadcast – server sends each message to ALL connected clients

How It Works (Architecture)

Server starts → waits for clients → Client 1 connects → Client 2 connects → Client 1 sends 'Hello' → Server receives it → Server broadcasts to Client 1 and Client 2 → Both see the message instantly.

Server Code (ChatServer.java)

```
import java.io.*;
import java.net.*;
import java.util.*;

/**
 * Chat Server - handles multiple clients simultaneously
 * Run this FIRST, then run ChatClient.java in separate terminals.
 */
public class ChatServer {

    static final int PORT = 5000;
    // Thread-safe list of all connected clients
    static List<PrintWriter> clients = Collections.synchronizedList(new ArrayList<>());

    public static void main(String[] args) throws IOException {
        System.out.println("Chat server started on port " + PORT);
        System.out.println("Waiting for clients...");

        // ■■ Listen for incoming connections ■■
        try (ServerSocket serverSocket = new ServerSocket(PORT)) {
            while (true) {
                Socket clientSocket = serverSocket.accept(); // Blocks until client connects
                System.out.println("New client connected: " + clientSocket.getInetAddress());

                // ■■ Give each client its own thread ■■
                Thread clientThread = new Thread(new ClientHandler(clientSocket));
                clientThread.start();
            }
        }
    }
}
```

```

    }
}

// ■■ Broadcast a message to ALL connected clients ■■
static void broadcast(String message) {
    synchronized (clients) {
        for (PrintWriter writer : clients) {
            writer.println(message); // Send to each client
        }
    }
}

// ■■ Handles one client connection ■■
static class ClientHandler implements Runnable {
    Socket socket;
    PrintWriter out;
    String username;

    ClientHandler(Socket socket) { this.socket = socket; }

    public void run() {
        try {
            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
            out = new PrintWriter(socket.getOutputStream(), true);

            // First message from client is their username
            out.println("Enter your username:");
            username = in.readLine();

            // Add this client to the broadcast list
            clients.add(out);
            broadcast("[SERVER] " + username + " has joined the chat!");

            // ■■ Listen for messages from this client ■■
            String message;
            while ((message = in.readLine()) != null) {
                if (message.equalsIgnoreCase("/quit")) break;
                broadcast(username + ": " + message);
            }

        } catch (IOException e) {
            System.out.println("Client disconnected: " + username);
        } finally {
            // Remove client when they disconnect
            clients.remove(out);
            broadcast("[SERVER] " + username + " has left the chat.");
            try { socket.close(); } catch (IOException e) {}
        }
    }
}
}

```

Client Code (ChatClient.java)

```

import java.io.*;
import java.net.*;
import java.util.Scanner;

/**

```

```

* Chat Client - connects to ChatServer
* Run multiple instances of this to simulate multiple users.
*/
public class ChatClient {

    static final String HOST = "localhost";
    static final int PORT = 5000;

    public static void main(String[] args) throws IOException {

        // ■■ Connect to server ■■
        Socket socket = new Socket(HOST, PORT);
        System.out.println("Connected to chat server!");

        BufferedReader in = new BufferedReader(
            new InputStreamReader(socket.getInputStream()));
        PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
        Scanner scanner = new Scanner(System.in);

        // ■■ Thread to receive messages from server ■■
        Thread receiveThread = new Thread(() -> {
            try {
                String serverMessage;
                while ((serverMessage = in.readLine()) != null) {
                    System.out.println(serverMessage); // Print messages from others
                }
            } catch (IOException e) {
                System.out.println("Disconnected from server.");
            }
        });
        receiveThread.setDaemon(true); // Stop when main thread stops
        receiveThread.start();

        // ■■ Send messages from keyboard ■■
        System.out.println("Type messages and press Enter. Type /quit to exit.");
        while (scanner.hasNextLine()) {
            String msg = scanner.nextLine();
            out.println(msg); // Send to server
            if (msg.equalsIgnoreCase("/quit")) break;
        }

        socket.close();
    }
}

```

How to Run (3 Terminal Windows)

```

# Terminal 1 - Start the server:
javac ChatServer.java
java ChatServer

# Terminal 2 - Start Client 1:
javac ChatClient.java
java ChatClient
# Enter username: Alice

# Terminal 3 - Start Client 2:
java ChatClient
# Enter username: Bob

```

```
# Now Alice and Bob can chat in real time!  
# Type /quit to disconnect.
```

TIP: You'll learn ServerSocket • Multi-threading • Broadcast • Client-Server architecture